Q13. PCA

Original Aim Apply PCA to the standardized numerical attributes of the Iris dataset to reduce them to two principal components. Train a classifier using the reduced features and report accuracy, precision, recall and F1-score.

Aim

Reduce standardized measurements to two principal components, train a classifier and report accuracy, precision, recall and F1 on test data.

Requirements

Python, Pandas, Matplotlib and scikit-learn's Pipeline, StandardScaler, PCA, LogisticRegression, splitting and metrics. References: [D12], [D14], [D15].

Essential theory

PCA is an unsupervised linear transformation that constructs orthogonal directions of greatest remaining training-data variance. Two principal components replace four standardized features with two scores per flower. A component is a weighted combination of the features, not one selected original column. Scaling prevents high-spread measurements from dominating the component directions. PCA maximizes retained variance rather than species separation, so high explained variance does not guarantee high classification accuracy.

For each class, precision=TP/(TP+FP), recall=TP/(TP+FN), and F1 is their harmonic mean. Weighted metrics average class-specific values using actual test-class supports as weights. Accuracy is the overall correct fraction. Weighted F1 is the weighted average of class F1 scores, not generally the harmonic mean of the already-averaged precision and recall.

Step-by-step procedure

1. Select the four numerical features and categorical Species target.
2. Perform a stratified 80/20 split with random_state=42.
3. Build StandardScaler -> PCA(n_components=2) -> LogisticRegression in one Pipeline; use full SVD for deterministic PCA on this small dataset.
4. Fit the Pipeline on training rows only, then predict test rows. Neither scaler nor PCA is refitted on test data.
5. Report accuracy and weighted precision/recall/F1 with zero_division=0 for defined fallback handling.
6. Inspect the training-fitted explained variance ratios, reduced shapes and confusion matrix; transform test rows with the fitted steps to make an optional diagnostic plot.

Complete Python program

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, confusion_matrix)
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
features = [
    "SepalLengthCm", "SepalWidthCm",
    "PetalLengthCm", "PetalWidthCm"
]
species = ["Iris-" + name for name in iris.target_names]
df = pd.DataFrame(iris.data, columns=features)
df["Species"] = pd.Categorical.from_codes(
    iris.target, categories=species
)
X = df[features]
y = df["Species"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
model = Pipeline([
    ("scale", StandardScaler()),
    ("pca", PCA(n_components=2, svd_solver="full")),
    ("classifier", LogisticRegression(solver="lbfgs", max_iter=1000))
])
model.fit(X_train, y_train)  # Every fit uses training rows only.
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
precision = precision_score(y_test, predictions,
                            average="weighted", zero_division=0)
recall = recall_score(y_test, predictions,
                      average="weighted", zero_division=0)
f1 = f1_score(y_test, predictions,
              average="weighted", zero_division=0)
print("Train/test shapes:", X_train.shape, X_test.shape)
print(f"Accuracy: {accuracy:.6f}")
print(f"Weighted precision: {precision:.6f}")
print(f"Weighted recall: {recall:.6f}")
print(f"Weighted F1: {f1:.6f}")
pca = model.named_steps["pca"]
print("Explained variance ratios:", pca.explained_variance_ratio_.round(6))
print(f"Total retained variance: {pca.explained_variance_ratio_.sum():.6f}")
print("Confusion matrix: rows actual, columns predicted; order:", species)
print(confusion_matrix(y_test, predictions, labels=species))
# Transform test rows using fitted training transformations, without refitting.
Z_test = pca.transform(model.named_steps["scale"].transform(X_test))
Z_train = pca.transform(model.named_steps["scale"].transform(X_train))
print("Reduced train/test shapes:", Z_train.shape, Z_test.shape)
fig, ax = plt.subplots(figsize=(7, 4.5))
colors = ["#147d92", "#dc8b26", "#7357a6"]
for label, color in zip(species, colors):
    mask = y_test.to_numpy() == label
    ax.scatter(Z_test[mask, 0], Z_test[mask, 1],
               label=label, color=color, s=42)
ax.set_xlabel("Principal component 1 (standardized-feature score)")
ax.set_ylabel("Principal component 2 (standardized-feature score)")
ax.set_title("Held-out Iris data in the training-fitted PCA space")
ax.legend(fontsize=8)
fig.tight_layout()
fig.savefig("q13_pca.png", dpi=180)
plt.show()
```

Code explanation

- Pipeline.fit trains each transformation only on training data before fitting the classifier on its two-component scores.
- Pipeline.predict applies the learned scaler and PCA to X_test, then uses the classifier. No test labels are passed to preprocessing.
- `average="weighted"` requests support weighting. All three classes have support 10 here, so weights are equal; that need not hold for other splits.
- `zero_division=0` assigns zero when a precision/recall denominator is undefined; it does not fix a poor model or alter the predictions.
- `named_steps` exposes the already fitted scaler/PCA. Their transform methods preserve the training-derived coordinate system. explained_variance_ratio_ refers to training-feature variance, not the fraction of correctly classified test rows.
- The plot colours points by their actual test species for interpretation only, after fitting; those labels were not used by PCA.

Verified output and figure

```
Train/test shapes: (120, 4) (30, 4)
Accuracy: 0.900000
Weighted precision: 0.902357
Weighted recall: 0.900000
Weighted F1: 0.899749
Explained variance ratios: [0.726772 0.230667]
Total retained variance: 0.957439
Confusion matrix: rows actual, columns predicted; order: ['Iris-setosa', 'Iris-versicolor',
  'Iris-virginica']
[[10  0  0]
 [ 0  9  1]
 [ 0  2  8]]
Reduced train/test shapes: (120, 2) (30, 2)
```



Observation

Training and test features reduce from (120,4)/(30,4) to (120,2)/(30,2). The first two components retain about 95.74% of the standardized training variance, but test accuracy is 90.00%, a separate quantity. Weighted precision is about 0.9024, recall 0.9000 and F1 0.8997. The matrix diagonal sums to 27 of 30. This split scores below Q11's four-feature classifier (28 of 30); that is one demonstration, not proof that PCA always reduces accuracy.

Result

A leakage-controlled scaler/PCA/classifier Pipeline reduced four inputs to two components and produced all four requested held-out classification metrics. Two components do not promise any fixed accuracy; evaluate the chosen representation for the actual task.

Common errors and troubleshooting

- Scaler/PCA fitted before the split: rebuild the Pipeline and fit only training rows; otherwise the evaluation leaks test information.
- Binary-average metric error: specify weighted for the three-class target.
- n_components too large: two is valid here; the retained component count cannot exceed the supported training-data rank/dimensions.
- Percent variance called accuracy: interpret explained_variance_ratio_ separately from accuracy_score.

Viva questions and answers

- What is a principal component? An orthogonal direction formed as a linear combination of centred input features.
- Does PCA use Species labels to find directions? No; it uses feature variance only.
- Why put PCA inside the Pipeline? To fit preprocessing on training data and apply it consistently to test rows.
- Can 95.74% retained variance imply 95.74% accuracy? No; variance preservation and classification correctness are different quantities.